

Отчет по лабораторной работе № 2

Генерация ER-диаграммы реляционной БД с помощью PlantUML и БЯМ

Автор: Дементьев Антон Павлович

Поток: ПИС 2.3

Группа: K3341

Ссылка на репозиторий проекта:

https://github.com/AntonDementiev/Information_Systems_Design_ITMO/tree/lab-2

Общая информация

Название работы: Генерация ER-диаграммы реляционной базы данных сервиса LegalAssist AI с использованием PlantUML и большой языковой модели

Цель работы: Сгенерировать ER-диаграмму реляционной БД информационной системы LegalAssist AI с помощью PlantUML и БЯМ, оценить результат, затем вручную расширить модель сущностями и связями и формализовать отношения для выбранных связей.

Задачи работы:

- Подготовить промпт для БЯМ, генерирующий PlantUML-скрипт ER-диаграммы (crow's foot)
- Получить скрипт и описание сущностей/связей от модели
- Проинтерпретировать ответ БЯМ в письменном виде
- Вручную дополнить диаграмму 2–3 новыми сущностями и связями
- Записать отношения для двух выбранных связей в обе стороны
- Визуализировать обе диаграммы в интерпретаторе PlantUML

1. Цель работы

Сгенерировать ER-диаграмму реляционной базы данных информационной системы LegalAssist AI с помощью скрипта PlantUML и большой языковой модели (БЯМ), оценить полученный результат, вручную расширить модель дополнительными сущностями и связями, а также формализовать отношения для выбранных связей в нотации «вороньи лапки» (crow's foot).

2. Постановка задачи

1. Подготовить промпт для БЯМ, генерирующий PlantUML-скрипт ER-диаграммы в нотации «вороньи лапки» (crow's foot) без противоречий предметной области LegalAssist AI. Промпт должен содержать указание дополнить диаграмму описанием сущностей и связей.
2. Получить PlantUML-скрипт и текстовое описание сущностей/связей от БЯМ.
3. Проинтерпретировать (оценить) ответ БЯМ в письменном виде.
4. Вручную дополнить диаграмму 2–3 новыми сущностями и соответствующими связями.
5. Записать отношения для двух выбранных связей в обе стороны.
6. Визуализировать обе диаграммы (исходную и расширенную) в интерпретаторе PlantUML.

3. Промпт для генерации скрипта

Ниже приведён промпт, направленный большой языковой модели (ChatGPT) для получения PlantUML-скрипта с описанием сущностей и связей:

Ты — аналитик данных и проектировщик реляционных БД. Предметная область: LegalAssist AI — автоматизированный юридический помощник, анализирующий правовые документы и дающий консультации на основе актуального законодательства.

Сгенерируй:

- 1) PlantUML-скрипт ER-диаграммы в нотации «вороньи лапки» (crow's foot), используя связи вида ||--o{, ||--||, |o--o{ и т.п.
- 2) Текстовое описание: перечисли сущности (назначение, ключевые атрибуты) и связи (кардинальности и смысл).

Требования к модели:

- Реляционная БД, нормализованная (минимум до 3НФ там, где уместно).
- Должны быть сущности: пользователи, загруженные документы, версии/метаданные документов, консультации/запросы пользователя, результаты анализа, источники права (законы/статьи/нормы) и ссылки (цитирования) на источники в результатах.
- У каждой сущности: PK, важные атрибуты с типами (INT, UUID, VARCHAR, TEXT, TIMESTAMP, BOOLEAN и т.п.).
- У связей: явные кардинальности. Если M:N — используй промежуточную таблицу.
- Добавь ограничения на уровне атрибутов в виде пометок (PK/FK/UNIQUE/NULL).
- Вывод дай в двух разделах: «PlantUML» и «Описание».

4. Полученный скрипт (ответ БЯМ)

Ниже представлен PlantUML-скрипт, сгенерированный моделью:

```
@startuml
hide circle
skinparam linetype ortho

entity "User" as user {
    * id : UUID <<PK>>
    --
    email : VARCHAR(255) <<UNIQUE>>
    full_name : VARCHAR(255)
    password_hash : VARCHAR(255)
    created_at : TIMESTAMP
    is_active : BOOLEAN
}
entity "Role" as role {
    * id : INT <<PK>>
    --
    code : VARCHAR(50) <<UNIQUE>>
    name : VARCHAR(100)
}
entity "UserRole" as user_role {
    * user_id : UUID <<PK, FK>>
    * role_id : INT <<PK, FK>>
    --
    granted_at : TIMESTAMP
}
entity "Document" as doc {
    * id : UUID <<PK>>
    --
    owner_user_id : UUID <<FK>>
    title : VARCHAR(255)
    doc_type : VARCHAR(50)
    uploaded_at : TIMESTAMP
    status : VARCHAR(50)
}
```

```

entity "DocumentVersion" as doc_ver {
  * id : UUID <<PK>>
  --
  document_id : UUID <<FK>>
  version_no : INT
  file_uri : VARCHAR(500)
  checksum : VARCHAR(128)
  created_at : TIMESTAMP
  extracted_text : TEXT
  <<UNIQUE(document_id, version_no)>>
}
entity "Consultation" as consult {
  * id : UUID <<PK>>
  --
  user_id : UUID <<FK>>
  created_at : TIMESTAMP
  channel : VARCHAR(50)
  problem_summary : TEXT
  status : VARCHAR(50)
}
entity "ConsultationAttachment" as consult_att {
  * consultation_id : UUID <<PK, FK>>
  * document_version_id : UUID <<PK, FK>>
  --
  attached_at : TIMESTAMP
}
entity "AnalysisResult" as analysis {
  * id : UUID <<PK>>
  --
  consultation_id : UUID <<FK>>
  model_name : VARCHAR(100)
  started_at : TIMESTAMP
  finished_at : TIMESTAMP
  risk_level : VARCHAR(20)
  conclusion : TEXT
  confidence : FLOAT
}
entity "LegalSource" as law_src {
  * id : UUID <<PK>>
  --
  source_type : VARCHAR(50) ' law/code/regulation/court_practice
  title : VARCHAR(500)
  jurisdiction : VARCHAR(100)
  effective_from : DATE
  effective_to : DATE
  updated_at : TIMESTAMP
}
entity "LegalProvision" as prov {
  * id : UUID <<PK>>
  --
  legal_source_id : UUID <<FK>>
  provision_ref : VARCHAR(100) ' e.g., Article 12, Clause 3
  text : TEXT
  valid_from : DATE
  valid_to : DATE
}
entity "Citation" as cite {
  * id : UUID <<PK>>
  --
  analysis_result_id : UUID <<FK>>
  legal_provision_id : UUID <<FK>>
  snippet : TEXT
  created_at : TIMESTAMP
  <<UNIQUE(analysis_result_id, legal_provision_id)>>
}
entity "Feedback" as fb {
  * id : UUID <<PK>>
  --
  user_id : UUID <<FK>>
  analysis_result_id : UUID <<FK>>
  rating : INT
}

```

```

comment : TEXT
created_at : TIMESTAMP
}

' Relationships (crow's foot)
user      ||--o{ doc      : owns
doc       ||--o{ doc_ver  : has
user      ||--o{ consult  : creates
consult   ||--o{ analysis : produces
consult   ||--o{ consult_att : has
doc_ver   ||--o{ consult_att : attached
law_src   ||--o{ prov     : contains
analysis  ||--o{ cite     : cites
prov      ||--o{ cite     : referenced_by
user      ||--o{ fb       : leaves
analysis  ||--o{ fb       : receives
role      ||--o{ user_role : assigned
user      ||--o{ user_role : has
@enduml

```

5. Диаграмма (визуализация скрипта в PlantUML)

Скрипт был вставлен в онлайн-интерпретатор PlantUML (plantuml.com). Полученная ER-диаграмма представлена ниже:

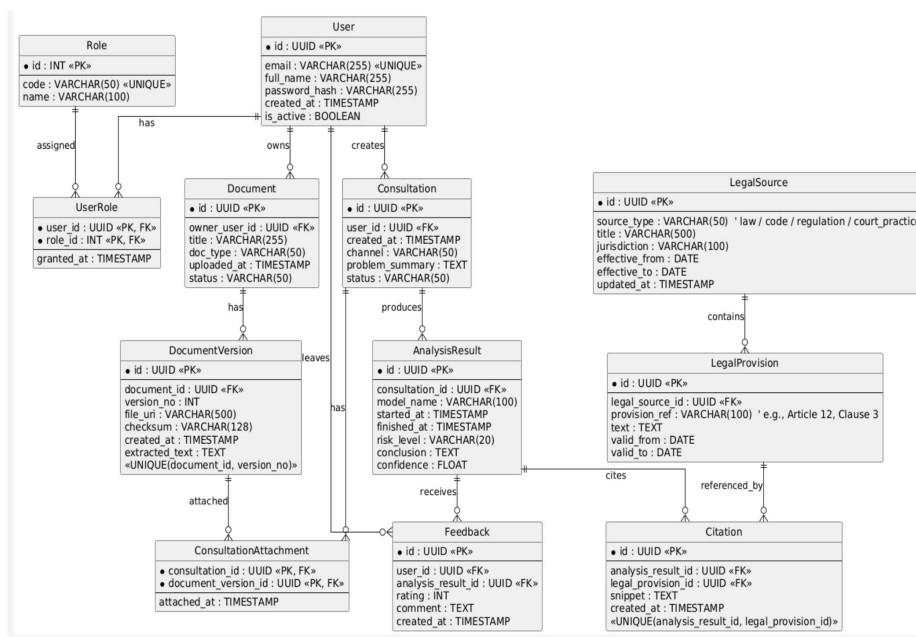


Рис. 1 — Исходная ER-диаграмма LegalAssist AI (генерация БЯМ)

6. Описание диаграммы

6.1. Состав сущностей и их назначение

User — учётная запись клиента или оператора системы. Хранит email (уникальный), хэш пароля и флаг активности.

Role / UserRole — справочник ролей и таблица назначения ролей пользователям. Связь M:N нормализована через промежуточную таблицу UserRole с составным первичным ключом.

Document — загруженный пользователем юридический документ (договор, иск, доверенность и т.д.). Хранит тип документа и статус обработки.

DocumentVersion — версии документа с извлечённым текстом и контрольной суммой. Составной уникальный индекс (document_id, version_no) гарантирует уникальность версий в рамках одного документа.

Consultation — обращение пользователя за консультацией; содержит канал обращения, краткое описание проблемы и статус.

ConsultationAttachment — таблица-связка M:N между Consultation и DocumentVersion: одна консультация может опираться на несколько версий документов.

AnalysisResult — результат анализа LLM по консультации: уровень риска, итоговый вывод и коэффициент уверенности модели.

LegalSource — источник права (закон, кодекс, регламент, судебная практика) с указанием юрисдикции и периода действия.

LegalProvision — конкретная норма (статья, пункт) внутри источника права с периодом действия.

Citation — таблица-связка M:N между AnalysisResult и LegalProvision: результат анализа ссылается на одну или несколько норм.

Feedback — оценка пользователем результата анализа (целочисленный рейтинг и текстовый комментарий).

6.2. Оценка ответа БЯМ

Сильные стороны. Модель корректно охватила ключевые процессы LegalAssist AI: пользователь → консультация → результат анализа → цитирование норм. Связи M:N правильно декомпозированы через промежуточные таблицы (UserRole, ConsultationAttachment, Citation). Предусмотрено версионирование документов с уникальным составным ключом. Типы данных подобраны адекватно предметной области.

Ограничения. Модель не предусмотрела диалоговый контекст (история сообщений чата) и журнал аудита, критически важный для юридических систем. Поля-перечисления (status, doc_type, risk_level) реализованы как VARCHAR вместо более строгих ENUM или справочников. Не предусмотрены подписки, тарифы и лимиты запросов.

Вывод. Схема непротиворечива предметной области и пригодна как основа для дальнейшего проектирования, однако требует расширения для реальной эксплуатации: добавление трекинга диалогов, аудита действий и управления тарифами.

7. Описание дополнения диаграммы

К исходной схеме вручную добавлены три сущности, покрывающие выявленные пробелы:

ChatSession — сессия диалога

Сущность фиксирует отдельный чат-сеанс в рамках консультации. Содержит атрибуты locale (язык сессии) и topic (тема). Необходима для многоходовых диалогов, когда одна консультация порождает несколько сессий.

- consultation_id : UUID <<FK>> — принадлежность к консультации

- locale : VARCHAR(10) — язык сессии (ru, en и т.д.)
- topic : VARCHAR(255) — краткая тема сессии

ChatMessage — сообщение диалога

Хранит каждое сообщение внутри сессии с указанием типа отправителя (user / assistant / system). Позволяет воспроизвести историю переписки, что критично для юридической трассируемости консультации.

- sender_type : VARCHAR(20) — тип отправителя (user / assistant / system)
- content : TEXT — текст сообщения
- sent_at : TIMESTAMP — метка времени отправки

AuditLog — журнал аудита

Фиксирует все значимые действия пользователей и системы: создание/изменение консультаций, загрузка документов, получение результатов анализа. Атрибут consultation_id допускает NULL — событие может быть не привязано к конкретной консультации (например, изменение профиля пользователя).

- actor_user_id : UUID <<FK>> — пользователь, выполнивший действие
- event_type : VARCHAR(50) — тип события (CREATE, UPDATE, DELETE и т.д.)
- entity_type / entity_id — тип и идентификатор затронутой сущности
- consultation_id : UUID <<FK, NULL>> — опциональная привязка к консультации

Добавленные связи

Новые связи в нотации crow's foot:

- Consultation ||--o{ ChatSession : has — одна консультация содержит одну или несколько чат-сессий
- ChatSession ||--o{ ChatMessage : contains — одна сессия содержит одно или несколько сообщений
- User ||--o{ AuditLog : writes — один пользователь порождает ноль или более записей аудита
- Consultation |o--o{ AuditLog : audited_in — события аудита могут быть опционально привязаны к консультации

8. Обновлённая диаграмма

Ниже представлены дополнительные фрагменты расширенного PlantUML-скрипта (новые сущности и связи добавлены к исходному скрипту из раздела 4):

```
' === Добавленные сущности ===
entity "ChatSession" as chat_sess {
    * id : UUID <<PK>>
    --
    consultation_id : UUID <<FK>>
    created_at : TIMESTAMP
    locale : VARCHAR(10)
    topic : VARCHAR(255)
}
entity "ChatMessage" as chat_msg {
    * id : UUID <<PK>>
```

```

--
chat_session_id : UUID <<FK>>
sender_type : VARCHAR(20) ' user/assistant/system
sent_at : TIMESTAMP
content : TEXT
}
entity "AuditLog" as audit {
  * id : UUID <<PK>>
  --
  actor_user_id : UUID <<FK>>
  event_time : TIMESTAMP
  event_type : VARCHAR(50)
  entity_type : VARCHAR(50)
  entity_id : UUID
  details : TEXT
  consultation_id : UUID <<FK, NULL>>
}

' === Добавленные связи ===
consult ||--o{ chat_sess : has
chat_sess ||--o{ chat_msg : contains
user ||--o{ audit : writes
consult |o--o{ audit : audited_in

```

Визуализация расширенного скрипта:

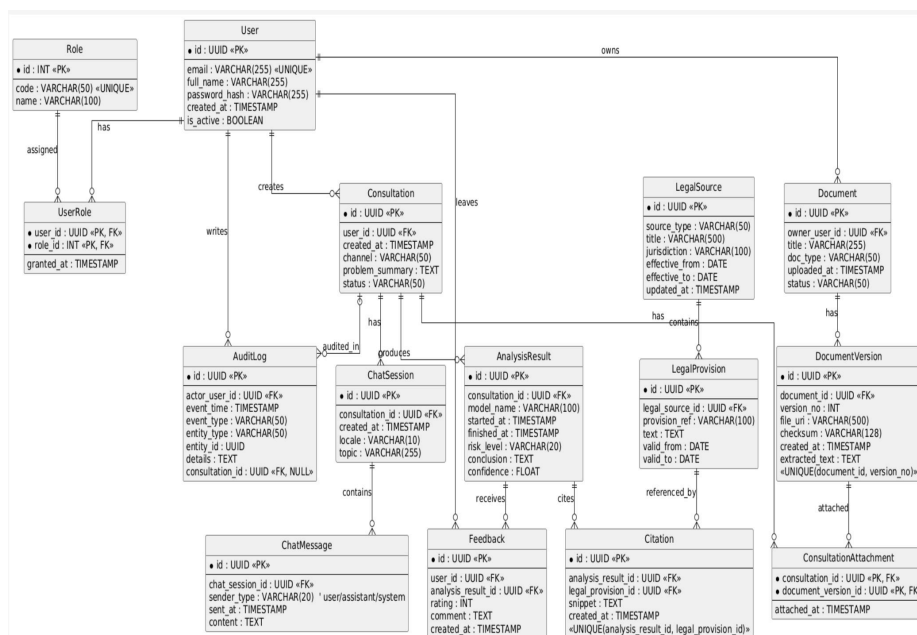


Рис. 2 — Расширенная ER-диаграмма LegalAssist AI (с добавленными сущностями)

9. Записанные отношения для двух связей

9.1. Связь User — Consultation

Со стороны User: Пользователь создаёт одну или несколько консультаций.

Со стороны Consultation: Каждая консультация создаётся только одним пользователем.

9.2. Связь Document — DocumentVersion

Со стороны Document: Документ имеет одну или несколько версий.

Со стороны DocumentVersion: Каждая версия принадлежит только одному документу.

10. Рефлексивное заключение

В ходе выполнения работы была успешно сгенерирована и расширена ER-диаграмма реляционной БД для LegalAssist AI.

Удалось:

- составить эффективный пром프트 и получить непротиворечивую схему от БЯМ
- критически оценить ответ модели и выявить архитектурные пробелы
- вручную дополнить диаграмму тремя сущностями (ChatSession, ChatMessage, AuditLog)
- формализовать отношения для двух связей в нотации crow's foot
- освоить работу с PlantUML как инструментом визуализации ER-диаграмм

Основные выводы:

БЯМ корректно восстановила «скелет» БД: документы, консультации, результаты анализа и ссылки на нормы права оказались покрыты без противоречий предметной области. Промпт с явными требованиями к нормализации, типам данных и ограничениям позволил получить качественный результат с первого запроса.

Вместе с тем генерация обнаружила типичный изъян «схемы по описанию»: модель не предусмотрела эксплуатационные сущности, критичные для продакшн-системы. История диалогов необходима для воспроизводимости консультации; журнал аудита — обязательное требование для юридически значимых систем.

Ручное дополнение трёх сущностей приблизило схему к промышленному стандарту. Работа подтвердила ценность итеративного подхода: генерация БЯМ как быстрый черновик, ручная доработка как аналитический контроль качества.

11. Подкаст (бонусный балл)

[Подкаст по Лабе 2](#)